

Empirical Evaluation Protocol for Human-Governed AI-Assisted Repair

A preregistration-ready benchmark design for diagnosis quality, repair correctness, verification, context ablation, security behavior, and human decision support

T. R. Bentley

Evaluation Protocol / Benchmark Design

September 2026

Publication status

This document defines the experiments required to make defensible performance claims. It intentionally reports no invented benchmark results. A subsequent results paper should populate the prespecified tables and analyses after the benchmark corpus and comparison systems are frozen.

Reference implementation: Tybent18/ai-debugger-pro

Current implementation languages: Python, C, C++, Java

Protocol principle: separate measured evidence from promotional claims

Abstract

AI Debugger Pro implements a human-governed repair loop in which deterministic execution evidence is interpreted by an optional language model, proposed source changes are previewed before mutation, user authorization is explicit, and approved repairs are rerun before being represented as verified. The architecture is technically inspectable, but architecture alone cannot establish whether the system diagnoses bugs accurately, repairs them reliably, reduces developer effort, calibrates confidence, benefits from exception-state or project context, or materially improves safety.

This protocol defines a reproducible empirical program for evaluating those questions. It specifies research questions, hypotheses, benchmark construction, baselines, dependent variables, pass/fail criteria, ablations, human-subject measures, security experiments, statistical analysis, reporting requirements, and reproducibility artifacts. The design deliberately distinguishes automated correctness metrics from human-factor outcomes and treats model/version/configuration as experimental variables rather than timeless properties of the system.

Keywords: program repair evaluation, AI debugging benchmark, LLM software engineering, human-AI collaboration, confidence calibration, sandbox evaluation, reproducibility

1. Evaluation Philosophy

The purpose of this benchmark is not to prove that an AI debugger is universally superior. It is to determine where the current architecture adds measurable value, where it fails, and which mechanisms cause observed improvements. Accordingly, every headline claim must map to an operational metric, a comparison condition, and a reproducible experimental procedure.

The implementation already contains a meaningful automated test surface covering AI suggestion handling, syntax analysis, demos, diagnosis objects, diffs, execution, history, project discovery, sandbox behavior, and interface theme behavior. The repository CI runs Ruff and pytest on Python 3.10 and 3.12. Those tests establish software-regression discipline; they are not substitutes for an external effectiveness benchmark.

2. Research Questions and Hypotheses

ID	Research question	Prespecified hypothesis
RQ1	How accurately does the system identify root causes?	H1: structured AI diagnosis will exceed deterministic error-text-only guidance on blinded root-cause scoring.
RQ2	How often does the first proposed repair produce a correct program?	H2: AI-assisted repair will improve pass@1 over no-repair and simple heuristic baselines.
RQ3	Does automatic rerun prevent false success claims?	H3: a nontrivial fraction of plausible approved patches will fail verification, demonstrating value in separating approval from verification.
RQ4	Does exception-state context improve Python diagnosis/repair?	H4: traceback locals will improve root-cause accuracy and/or pass@1 on state-dependent bugs.
RQ5	Does bounded project context improve multi-file failures?	H5: project context will improve multi-file tasks but may add latency/token cost.
RQ6	Is model confidence calibrated?	H6: raw confidence will show measurable calibration error and should not be treated as correctness probability without calibration.
RQ7	Does the approval interface improve human decisions?	H7: diff preview plus structured diagnosis will improve correct accept/reject decisions relative to prose-only repair suggestions.
RQ8	What security properties are actually supported?	H8: documented Docker controls will block the tested network/resource/privilege behaviors, while results will not be generalized to absolute isolation.

3. Benchmark Corpus

The corpus should be frozen before primary evaluation and assigned a public version identifier. Tasks must contain an executable oracle: tests, expected output, compiler outcome, or another machine-checkable specification. The benchmark should include both synthetic controlled defects and real defects drawn from permissively redistributable repositories or established public bug datasets.

3.1 Stratification

Dimension	Required strata
Language	Python, C, C++, Java; report per-language and pooled results.
Failure stage	Parse/syntax, compilation, startup/import, runtime exception, incorrect output, multi-file integration.
Bug family	Name/scope, type/shape, boundary/index, null/None, arithmetic, API misuse, resource/state, control flow, data structure, dependency/interface.
Context need	Single-line/local, single-file nonlocal, exception-state dependent, multi-file dependent.
Difficulty	Low, medium, high based on prespecified structural criteria rather than model success.
Repair size	1-line, 2-5 lines, >5 lines or multi-location.
Origin	Controlled mutation vs. real-world defect.

A publication-scale first study should target at least 400 tasks, with enough cases in each language and major bug family to support stratified estimates. If resource constraints require a smaller pilot, it must be labeled exploratory and should not be used for strong comparative claims.

3.2 Leakage and Contamination Controls

Where possible, use newly authored controlled defects, post-training-time defects, or private holdout tasks until evaluation is complete. For public historical bugs, record repository, commit, date, issue/PR linkage, and whether the fix is likely to appear in model training data. Results should be reported separately for low- and high-contamination-risk subsets.

3.3 Oracle Requirements

A patch is not correct merely because the original failure disappears. The oracle must detect functional correctness and regressions. Each task should therefore include the original failing condition plus unaffected behavior. For real projects, use the project's test suite when feasible; for controlled tasks, create hidden tests that cover both the defect and preserved behavior.

4. Experimental Conditions

Condition	Purpose
C0: Raw evidence only	Compiler/runtime/traceback output; establishes non-AI baseline.
C1: Offline deterministic guidance	Measures value of the application's non-model fallback.
C2: AI + error output	Tests structured AI diagnosis without enriched runtime state.
C3: AI + exception locals	Python ablation for captured traceback state.
C4: AI + bounded project context	Tests multi-file contextual benefit.
C5: Full AI Debugger Pro protocol	Diagnosis + context where applicable + diff + approval + rerun + regression suggestion.

Condition	Purpose
External comparator(s)	One or more contemporary coding assistants/debuggers, version-frozen and configured according to documented defaults.

All model-assisted conditions must record provider, exact model identifier, date, temperature, token limits, prompt template hash, and retry policy. If the model is updated during the study, the affected runs constitute a new condition rather than silently replacing prior measurements.

5. Primary Outcome Measures

Metric	Definition	Primary use
Root-cause accuracy	Blinded ordinal/exact scoring against a gold explanation.	Diagnosis quality
Pass@1 repair	First proposed repair passes all hidden/public oracle tests.	Primary repair endpoint
Compile/run recovery	Patch removes original syntax/compile/runtime failure.	Secondary; insufficient alone
Regression-free rate	Repair passes preserved-behavior tests.	Repair safety
False-verification rate	System labels verified when oracle still fails.	Critical integrity metric
Repair precision	Accepted/applied repairs that are oracle-correct.	Human/system decision quality
Repair recall	Oracle-repairable tasks resulting in correct applied repair.	Coverage
Edit distance / patch size	Magnitude of source mutation relative to gold/minimal repair.	Change discipline
Latency	Wall-clock diagnosis and verified-repair time.	Efficiency
Token/cost footprint	Input/output tokens and provider cost per task.	Operational efficiency
Calibration error	Gap between confidence and empirical correctness.	Confidence validity

The primary automated endpoint should be pass@1 against the complete oracle. Root-cause accuracy should be a co-primary endpoint if diagnosis quality is a central publication claim. Compiler recovery alone must never be reported as repair correctness.

6. Root-Cause Scoring Protocol

Two independent reviewers, blinded to condition and system identity, should score each diagnosis using a prespecified rubric: 0 = incorrect or irrelevant; 1 = identifies symptom only; 2 = partially identifies causal mechanism; 3 = correct root cause but incomplete localization/detail; 4 = correct cause, localization, and relevant mechanism. Disagreements should be adjudicated by a third reviewer. Inter-rater reliability must be reported before adjudication.

For tasks with objective structured causes, an additional exact-category score should be used. Free-text semantic scoring by another LLM may be included as a secondary analysis but should not replace blinded human scoring without validating agreement.

7. Repair Correctness Protocol

Each candidate patch is applied to a clean task checkout and evaluated in an isolated environment. The harness records parse/compile outcome, original failing test, full hidden/public test suite, timeout, exit status, changed files, changed lines, and execution logs. A repair is correct only if all required oracle checks pass.

For the AI Debugger Pro full-protocol condition, separately record proposal correctness, human approval decision, rerun outcome, and oracle outcome. This permits analysis of where the repair pipeline succeeds or fails rather than reducing the entire process to one final percentage.

8. Context Ablation

Python state-dependent tasks should be run with identical source/error prompts both with and without captured traceback locals. Multi-file tasks should similarly compare error-only context with bounded project context. Pairing the same task across conditions controls for task difficulty. Measure accuracy, pass@1, latency, tokens, and context size.

Ablation is essential because richer context can help for the wrong reason—for example by leaking the expected answer—or hurt through distraction. Context should therefore be treated as an intervention, not automatically as a benefit.

9. Confidence Calibration

The Diagnosis contract exposes a confidence value bounded between 0 and 1. The study should test whether that value tracks empirical correctness. Bin predictions by confidence and report reliability diagrams, expected calibration error, Brier score, and accuracy at prespecified confidence thresholds. Calibration should be evaluated separately for root-cause correctness and patch correctness.

Confidence must not be used to automate approval in the primary study. A later policy experiment may test whether calibrated thresholds can safely reduce human review, but only after calibration is measured on held-out data.

10. Human-Subject Evaluation

A separate controlled study should evaluate the approval-gated interface. Participants should be stratified by programming experience and assigned debugging tasks under at least two conditions: structured diagnosis + code proposal without a diff/approval workflow, and the full AI Debugger Pro preview/approval/verification workflow.

Measure	Definition
Decision accuracy	Correctly approve oracle-correct proposals and reject incorrect proposals.
Time to decision	Time from proposal display to approve/reject action.
Verified repair time	Time from initial failure to oracle-correct verified repair.
Overreliance	Rate of approving incorrect high-confidence proposals.
Underreliance	Rate of rejecting correct proposals.
Comprehension	Post-task explanation of defect and repair mechanism.
Workload	Standardized workload instrument or prespecified Likert battery.
Trust calibration	Difference between stated trust and observed system correctness.

Human-subject work should receive the appropriate institutional/ethical review for the intended publication venue. Participant consent, compensation, exclusion criteria, sample-size justification, and anonymization must be documented before data collection.

11. Security Evaluation

Security evaluation should test the documented controls rather than attempt to prove universal containment. The benchmark should include benign adversarial programs designed to attempt network access, excessive memory allocation, process proliferation, writes outside permitted temporary/workspace paths, privilege escalation primitives, and access to intentionally planted host-only environment secrets.

Success criteria should map directly to configuration: network attempts fail; resource abuse is bounded/terminated; forbidden writes fail; capability-dependent operations fail; and planted host secrets are absent from the executed environment. Any escape or host impact is a critical finding and should halt unsafe test classes until contained.

Security testing should occur only in disposable, isolated research infrastructure. The paper should report the tested threat model and Docker/runtime versions and explicitly state that passing these tests does not establish absolute sandbox security.

12. Statistical Analysis Plan

For paired binary outcomes such as pass@1 across two conditions, use paired proportion methods such as McNemar's test with effect sizes and confidence intervals. For multiple model/context conditions, use mixed-effects logistic

regression with task as a random effect and language, bug family, difficulty, and condition as fixed effects where assumptions permit. Ordinal root-cause scores can use appropriate ordinal mixed models or prespecified nonparametric paired tests.

Report absolute differences, relative effects where meaningful, 95% confidence intervals, and exact sample counts. Correct for multiple comparisons across secondary hypotheses using a prespecified method. Do not infer practical significance from p-values alone.

13. Sample Size and Power

Final sample size should be determined from a preregistered minimum detectable effect. As an illustrative planning target—not a claimed power calculation—a 400-task paired corpus provides substantially more stable estimates than a small demo suite and permits meaningful per-language stratification. A formal power analysis should use pilot estimates of discordant repair outcomes for the planned paired tests.

For the human study, power should be based on the primary decision-accuracy or time-to-verified-repair endpoint and account for repeated tasks per participant. Pilot data should be used only to estimate variance/effect size; pilot participants should be excluded from confirmatory analysis if the protocol changes materially.

14. Reproducibility Package

Artifact	Required contents
Benchmark manifest	Task IDs, provenance, language, strata, contamination risk, oracle metadata.
Frozen task bundles	Source at buggy revision plus tests/specification; licenses preserved.
Runner	Containerized evaluation harness with fixed time/resource limits.
Prompt manifest	Exact system/user templates and cryptographic hashes.
Model manifest	Provider, model ID, date, parameters, retry policy.
Environment manifest	OS, Docker, compilers/JDK/Python, dependency lock data.
Raw outputs	Diagnoses, proposed patches, execution logs, timing, token counts, outcomes.
Analysis code	Scripts/notebooks that reproduce every reported table and figure.
Exclusion log	Every excluded task/run with prespecified reason.
Result checksum	Hashes tying published results to raw artifacts.

The repository's existing CI already runs Ruff and pytest on Python 3.10 and 3.12. The benchmark harness should add a separate reproducibility workflow rather than conflating unit-test CI with scientific evaluation.

15. Prespecified Result Tables

Table	Required reporting
T1	Corpus composition by language, bug family, failure stage, difficulty, origin.
T2	Pass@1 by condition with counts, percentages, confidence intervals.
T3	Root-cause score distribution and inter-rater reliability.
T4	Regression-free repair rate and false-verification rate.
T5	Context ablations: effect size, latency, token/context cost.
T6	Confidence calibration metrics and threshold performance.
T7	Human approval accuracy, over/underreliance, time, workload.
T8	Security-control test matrix and observed outcome.
T9	Per-language/per-bug-family subgroup results.
T10	Failure taxonomy for incorrect diagnoses and repairs.

16. Failure Taxonomy

Every incorrect repair should be classified after primary scoring. Suggested categories are: wrong root cause; correct cause/wrong localization; incomplete patch; syntax/compile regression; semantic regression; overbroad rewrite; hallucinated API/dependency; failure to preserve intent; timeout/resource failure; context omission; context distraction; and evaluation-infrastructure failure. This taxonomy converts benchmark failures into an engineering roadmap.

17. Publication Claim Matrix

Potential claim	Minimum evidence required
'Improves repair accuracy'	Statistically and practically meaningful pass@1 improvement against prespecified baseline(s) on frozen corpus.
'Reduces debugging time'	Human or controlled workflow timing with comparable task assignments.
'Exception state improves diagnosis'	Paired ablation showing root-cause/repair improvement with captured locals.
'Project context improves multi-file repair'	Paired multi-file ablation with benefit exceeding added cost/latency tradeoff.
'Confidence is useful'	Calibration metrics plus threshold utility on held-out data.
'Approval gating improves safety'	Human decision study showing fewer incorrect applied repairs or better accept/reject calibration.
'Verification prevents false success'	Observed cases where plausible/approved repairs fail rerun/oracle, plus measured false-verification rate.
'Sandbox is secure'	Do not make this absolute claim; report only tested controls and threat model.

18. Threats to Validity

External validity. A four-language benchmark does not represent all languages, frameworks, or production environments. **Model drift.** Hosted models change; exact identifiers and dates are therefore mandatory. **Benchmark contamination.** Public bugs may exist in training data. **Oracle incompleteness.** Passing tests may still conceal defects. **Human sampling.** Student, hobbyist, and professional developers may interact with approval gates differently. **Instrumentation effects.** Added tracing/context can alter timing and, in some languages, behavior. **Comparator fairness.** Competing tools must receive equivalent task information and reasonable configuration.

19. Preregistration Checklist

- Freeze before confirmatory runs
- Benchmark version and task inclusion/exclusion rules
- Primary and secondary endpoints
- Conditions, models, prompts, and context policies
- Sample-size/power calculation
- Statistical tests and multiplicity correction
- Human-study recruitment and exclusion criteria
- Security threat model and stop conditions
- Handling of provider errors/timeouts/retries
- Contamination-risk classification
- Subgroup analyses designated confirmatory vs. exploratory
- Public artifact/release plan

20. Conclusion

AI Debugger Pro is now mature enough architecturally to justify a serious evaluation, but not to skip one. The strongest next move is to convert the project's visible engineering features into falsifiable hypotheses: whether structured diagnosis

is accurate, whether repairs pass complete oracles, whether context helps, whether confidence is calibrated, whether human approval improves decisions, whether rerun verification catches plausible failures, and whether the documented execution restrictions behave as intended.

A favorable benchmark would support publication-level claims with evidence. An unfavorable benchmark would be equally valuable because its failure taxonomy would identify which components require redesign. In both cases, the protocol protects the project from a common failure mode in AI tooling: presenting an impressive demonstration as if it were an empirical result.

References

1. Bentley, T. R. *AI Debugger Pro: A Multi-Language Debugging Framework with LLM-Assisted Error Interpretation and Execution Tracing*. 2026.
2. Bentley, T. R. *From Stateless Execution to Human-Governed AI Repair: The Evolution of AI Debugger Pro, Versions 0-6*. 2026.
3. Bentley, T. R. *AI Debugger Pro: Current Architecture and Human-Governed Repair Protocol*. 2026.
4. Tybent18/ai-debugger-pro. Current source, tests, and CI configuration, accessed September 2026.

Document role: preregistration-ready evaluation blueprint. No performance values in this paper should be interpreted as observed results unless explicitly identified as future populated measurements.